🐍 Python Collections Complete Notes (With Java Comparison)

🌟 What are Collections in Python?

Collections are special data types used to store multiple values in a single variable.

Instead of creating many variables:

```
name1 = "Rahul"
name2 = "Ravi"
name3 = "Anu"
```

We use collections:

```
names = ["Rahul", "Ravi", "Anu"]
```

👉 Collections help in:

- Storing multiple values
- Managing data easily
- Faster operations
- Better code organization

📚 Main Collections in Python

Collection	Ordered	Mutable	Duplicates Allowed	Syntax
List	✅ Yes	✅ Yes	✅ Yes	`[]`
Tuple	✅ Yes	❌ No	✅ Yes	`()`
Set	❌ No	✅ Yes	❌ No	`{}`
Dictionary	✅ Yes	✅ Yes	Keys ❌ Values ✅	`{key:value}`

☕ Java Collection Hierarchy

```
Collection Interface
│
├── List Interface
│   ├── ArrayList
│   ├── LinkedList
│   └── Vector
│
├── Set Interface
│   ├── HashSet
│   ├── LinkedHashSet
│   └── TreeSet
│
├── Queue Interface
│   ├── PriorityQueue
│   └── ArrayDeque
│
└── Map Interface
    ├── HashMap
    ├── LinkedHashMap
    └── TreeMap
```

🐍 Python Collection Structure

Python does not use interfaces like Java.

But internally collections are categorized like this:

```
Collections
│
├── List
├── Tuple
├── Set
└── Dictionary
```

👉 Python is simpler compared to Java.

🔥 1️⃣ LIST

📘 What is List?

A List is:

✅ Ordered

✅ Mutable (changeable)

✅ Allows duplicates

🧠 Java Equivalent

```
ArrayList
```

✍️ Syntax

```
list_name = [values]
```

✅ Example

```
numbers = [10, 20, 30, 40]
print(numbers)
```

📍 Accessing Elements

☕ Java

```
list.get(0)
```

🐍 Python

```
numbers[0]
```

✏️ Modifying List

```
numbers[1] = 200
print(numbers)
```

➕ Adding Elements

☕ Java

```
list.add(50)
```

🐍 Python

```
numbers.append(50)
```

📥 Inserting at Specific Position

☕ Java

```
list.add(1, 100)
```

🐍 Python

```
numbers.insert(1, 100)
```

❌ Removing Elements

☕ Java

```
list.remove(20)
```

🐍 Python

```
numbers.remove(20)
```

🗑️ Remove Using Index

☕ Java

```
list.remove(0)
```

🐍 Python

```
numbers.pop(0)
```

📏 Length

☕ Java

```
list.size()
```

🐍 Python

```
len(numbers)
```

🔄 Sorting

☕ Java

```
Collections.sort(list)
```

🐍 Python

```
numbers.sort()
```

🔃 Reverse

```
numbers.reverse()
```

🧪 List Methods

Python Method	Purpose	Java Equivalent
append()	Add item	add()
insert()	Insert item	add(index,value)
remove()	Remove value	remove()
pop()	Remove using index	remove(index)
sort()	Sort list	Collections.sort()
reverse()	Reverse list	Collections.reverse()
clear()	Remove all	clear()
count()	Count value	Collections.frequency()

Python Method	Purpose	Java Equivalent
index()	Find index	indexOf()

⚠️ Edge Cases in List

Empty List

```
empty = []
```

Mixed Datatypes Allowed

```
data = [10, "Hello", True, 5.5]
```

👉 Unlike Java, Python lists can store different datatypes.

List Allows Duplicates

```
nums = [1, 1, 1, 2]
```

🔥 2️⃣ TUPLE

📘 What is Tuple?

Tuple is:

✅ Ordered

❌ Immutable

✅ Allows duplicates

🧠 Java Equivalent

No exact equivalent.

Closest:

- Immutable List
- Fixed data structure

✍️ Syntax

```
tuple_name = (values)
```

✅ Example

```
numbers = (10, 20, 30)
print(numbers)
```

📍 Accessing Elements

```
print(numbers[0])
```

❌ Cannot Modify Tuple

```
numbers[0] = 100
```

👉 Error because tuple is immutable.

🧪 Tuple Methods

Method	Purpose	Java Equivalent
count()	Count occurrences	frequency()
index()	Find index	indexOf()

👉 Tuple has fewer methods because it is immutable.

⚠️ Important Edge Cases in Tuple

Single Value Tuple

```
t = (5)
print(type(t))
```

Output

```
<class 'int'>
```

👉 Python considers it as integer.

✅ Correct Single Element Tuple

```
t = (5,)
print(type(t))
```

Output

```
<class 'tuple'>
```

👉 Comma is compulsory.

Empty Tuple

```
empty = ()
```

Tuple Packing

```
data = 10, 20, 30
```

Tuple Unpacking

```
a, b, c = data

print(a)
print(b)
print(c)
```

🔥 3️⃣ SET

📘 What is Set?

Set is:

❌ Unordered

✅ Mutable

❌ No duplicates

🧠 Java Equivalent

```
HashSet
```

✍️ Syntax

```
set_name = {values}
```

✅ Example

```
numbers = {10, 20, 30}
print(numbers)
```

⚠️ Duplicates Automatically Removed

```
nums = {1, 1, 1, 2}
print(nums)
```

Output

```
{1, 2}
```

➕ Add Element

☕ Java

```
set.add(10)
```

🐍 Python

```
numbers.add(10)
```

❌ Remove Element

☕ Java

```
set.remove(10)
```

🐍 Python

```
numbers.remove(10)
```

🔄 Set Operations

Union

☕ Java

```
set1.addAll(set2)
```

🐍 Python

```
set1.union(set2)
```

Intersection

☕ Java

```
set1.retainAll(set2)
```

🐍 Python

```
set1.intersection(set2)
```

Difference

🐍 Python

```
set1.difference(set2)
```

🧪 Set Methods

Method	Purpose	Java Equivalent
add()	Add element	add()
remove()	Remove element	remove()
union()	Combine sets	addAll()
intersection()	Common values	retainAll()
difference()	Unique values	removeAll()
clear()	Remove all	clear()

⚠️ Important Edge Cases in Set

Empty Set Problem

```
s = {}
print(type(s))
```

Output

```
<class 'dict'>
```

👉 Empty curly braces create dictionary.

✅ Correct Empty Set

```
s = set()
```

Set Does Not Support Indexing

```
nums = {1, 2, 3}

# nums[0]
```

👉 Error because set is unordered.

🔥 4️⃣ DICTIONARY

📘 What is Dictionary?

Dictionary stores data in:

```
key : value
```

It is:

✅ Ordered

✅ Mutable

❌ Duplicate keys not allowed

🧠 Java Equivalent

```
HashMap
```

✍️ Syntax

```
dict_name = {
    key:value
}
```

✅ Example

```
student = {
    "name": "Rahul",
    "age": 21
}

print(student)
```

📍 Accessing Values

☕ Java

```
map.get("name")
```

🐍 Python

```
student["name"]
```

➕ Adding / Updating

```
student["city"] = "Bangalore"
```

❌ Removing

☕ Java

```
map.remove("name")
```

🐍 Python

```
student.pop("name")
```

🧪 Dictionary Methods

Method	Purpose	Java Equivalent
keys()	Get all keys	keySet()
values()	Get all values	values()
items()	Get key-value pairs	entrySet()
get()	Get value safely	get()
pop()	Remove key	remove()
clear()	Remove all	clear()
update()	Update dictionary	putAll()

⚠️ Important Edge Cases in Dictionary

Empty Dictionary

```
d = {}
```

Duplicate Keys Not Allowed

```
data = {
    "a": 1,
    "a": 2
}

print(data)
```

Output

```
{'a': 2}
```

👉 Latest value overwrites previous value.

Dictionary Keys Must Be Immutable

✅ Allowed:

```
{
    1: "Hello",
    "name": "Rahul",
    (1,2): "Tuple"
}
```

❌ Not Allowed:

```
{
    [1,2]: "List"
}
```

👉 Lists are mutable.

🔥 Python Collection Comparison

Feature	List	Tuple	Set	Dictionary
Ordered	✅	✅	❌	✅
Mutable	✅	❌	✅	✅
Duplicates	✅	✅	❌	Keys ❌
Indexing	✅	✅	❌	Using keys
Syntax	`[]`	`()`	`{}`	`{k:v}`

☕ Java vs 🐍 Python Collections

Java	Python
ArrayList	List
HashSet	Set
HashMap	Dictionary
Immutable List	Tuple

🚀 When to Use What?

Collection	Best Use
List	Frequently changing data
Tuple	Fixed data
Set	Unique values
Dictionary	Key-value data

🎯 Example Programs

List Example

```
nums = [10, 20, 30]
nums.append(40)

print(nums)
```

Tuple Example

```
data = (1, 2, 3)

print(data[0])
```

Set Example

```
nums = {1, 1, 2, 3}

print(nums)
```

Dictionary Example

```
student = {
    "name": "Rahul",
    "age": 21
}

print(student["name"])
```

🌟 Final Summary

List

- Ordered
- Mutable
- Duplicates allowed

Tuple

- Ordered
- Immutable
- Faster than list

Set

- Unordered
- Unique values only

Dictionary

- Key-value pair storage
- Fast searching using keys

🧠 Most Important Interview Points

✅ Lists are mutable

✅ Tuples are immutable

✅ Sets remove duplicates

✅ Dictionaries store key-value pairs

✅ `{}` creates dictionary, not set

✅ `(5)` is int, `(5,)` is tuple

✅ Set does not support indexing

✅ Dictionary keys must be immutable